Artificial and Computational Intelligence

AIMLCZG557

BITS Pilani

Pilani | Dubai | Goa | Hyderabad

ACI Team

innovate achieve lead

BITS Pilani

Pilani|Dubai|Goa|Hyderabad

AIMLCZG557 – CS#8
Alpha-Beta Pruning and Monte Carlo Tree Search

Agenda for CS #8

1) Recap of CS #7
2) Complete Static Evaluation
3) Introduction to Alpha-Beta Pruning
4) Monte Carlo Tree Search
5) Details on Mid-Semester Examination
6) Q&A!

Course Plan: Where are we in the Syllabus?

M1	Introduction to AI
M2	Problem Solving Agent using Search
M3	Game Playing
M4	Knowledge Representation using Logics
M5	Probabilistic Representation and Reasoning
M6	Reasoning over time
M7	Ethics in AI

Alpha – Beta Pruning!

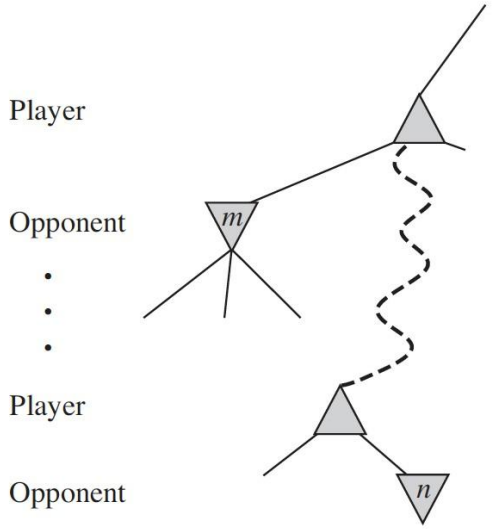
innovate achieve lead

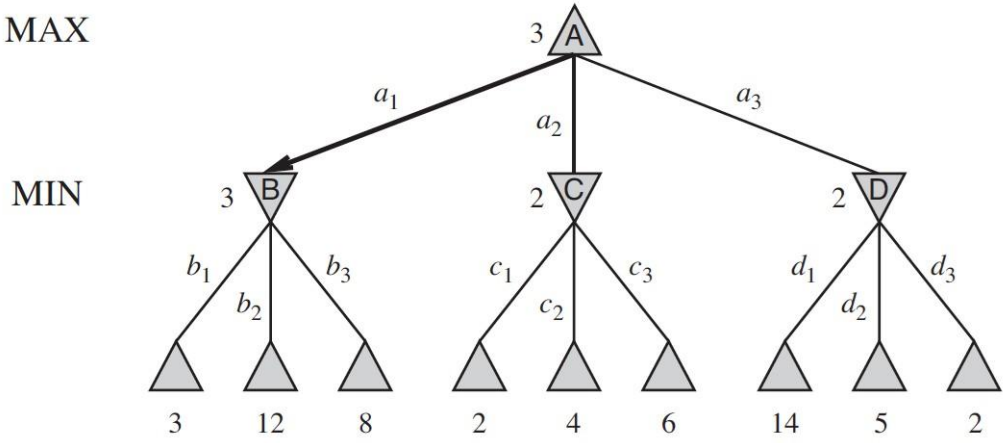
- **Alpha–beta ($\alpha-\beta$)** algorithm was discovered independently by a few researches in mid 1900s. Alpha–beta is actually an improved minimax using a heuristic. It stops evaluating a move when it makes sure that it's worse than previously examined move. Such moves need not to be evaluated further.
- When added to a simple minimax algorithm, it gives the same output, but cuts off certain branches that can't possibly affect the final decision - dramatically improving the performance.
- The main concept is to maintain two values through whole search:
  - **Alpha**: Best already explored option for player Max
  - **Beta**: Best already explored option for player Min
- Initially, alpha is negative infinity and beta is positive infinity, i.e. in our code we'll be using the worst possible scores for both players.

Alpha – Beta Pruning

General Principle:

At a node ***n*** if a player has better option at the parent of ***n*** or further up, then ***n*** node will never be reached .Hence the entire subtree from node ***n*** can be pruned





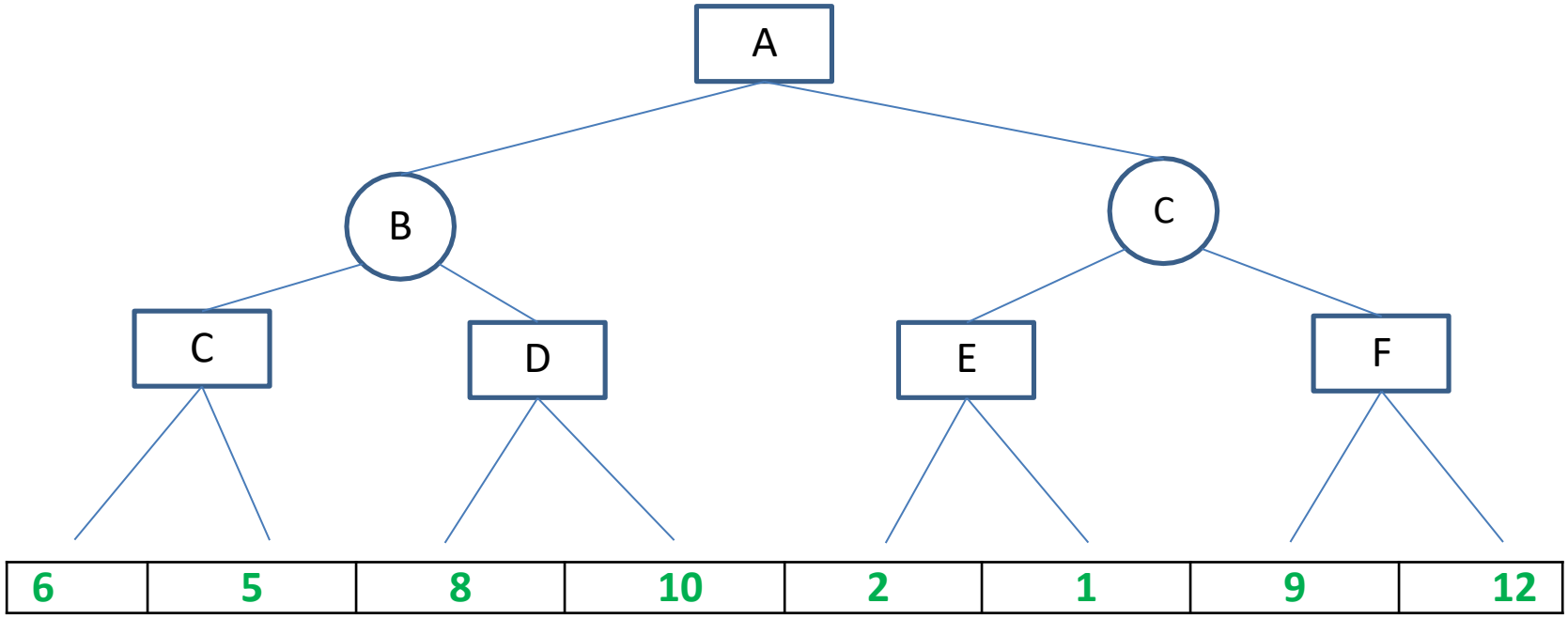
$$\begin{aligned}\text{MINIMAX}(root) &= \max(\min(3,12,8),\min(2,x,y),\min(14,5,2)) \\ &= \max(3,\min(2,x,y),2) \\ &= \max(3,z,2) \qquad \text{where } z = \min(2,x,y) \le 2 \\ &= 3.\end{aligned}$$

Alpha – Beta Pruning

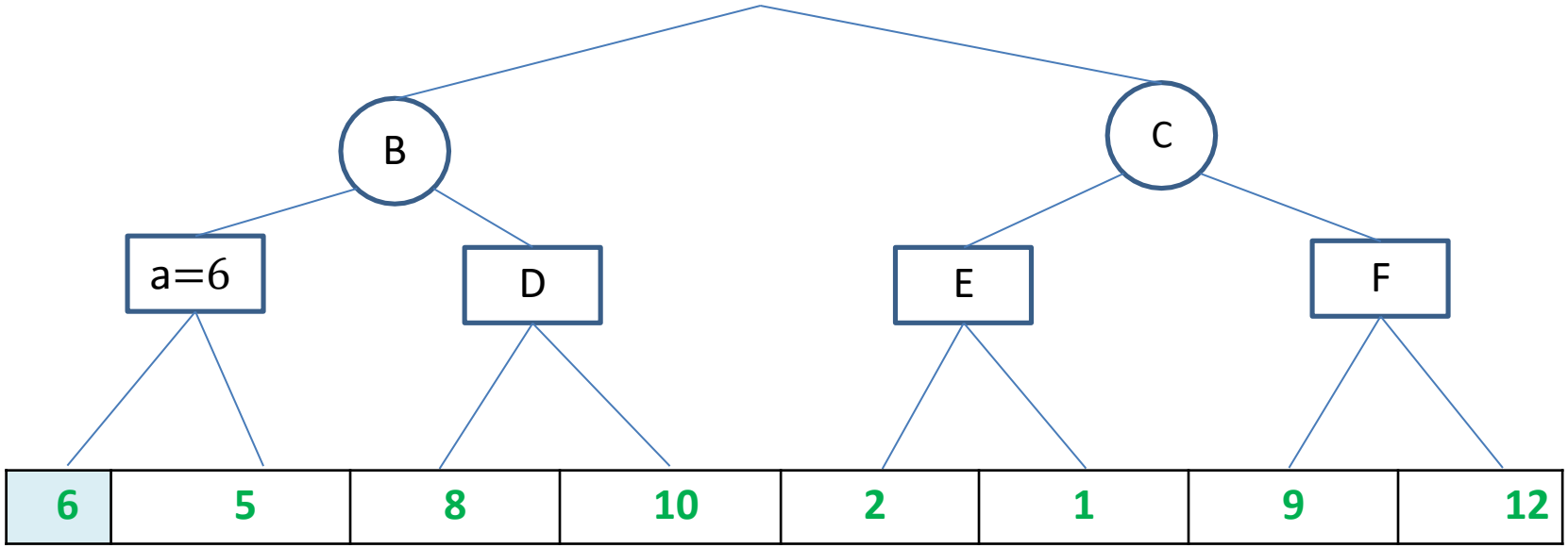
Steps in Alpha – Beta Pruning:

1. At root initialize alpha = -∞ and beta = + ∞. This is to set the worst-case boundary to start the algorithm which aims to increase alpha and decrease beta as much as optimally possible
2. Navigate till the depth / limit specified and get the static evaluated numeric value.
3. For every value VAL being analyzed : Loop till all the leaf/terminal/specified state level nodes are analyzed & accounted for OR until **beta <= alpha.**
    1. If the player is MAX :
        1. If VAL > alpha
        2. then reset alpha = VAL
        3. also check **if** beta <=alpha **then** tag the path as unpromising (TO BE AVOIDED) **and** prune the branch from game tree. Rest of their siblings are not considered for analysis
    2. Else if the player is MIN:
        1. If VAL < beta
        2. then reset beta = VAL
        3. also check **if** beta <=alpha **then** tag the path as unpromising (TO BE AVOIDED) **and** prune the branch from game tree. Rest of their siblings are not considered for analysis

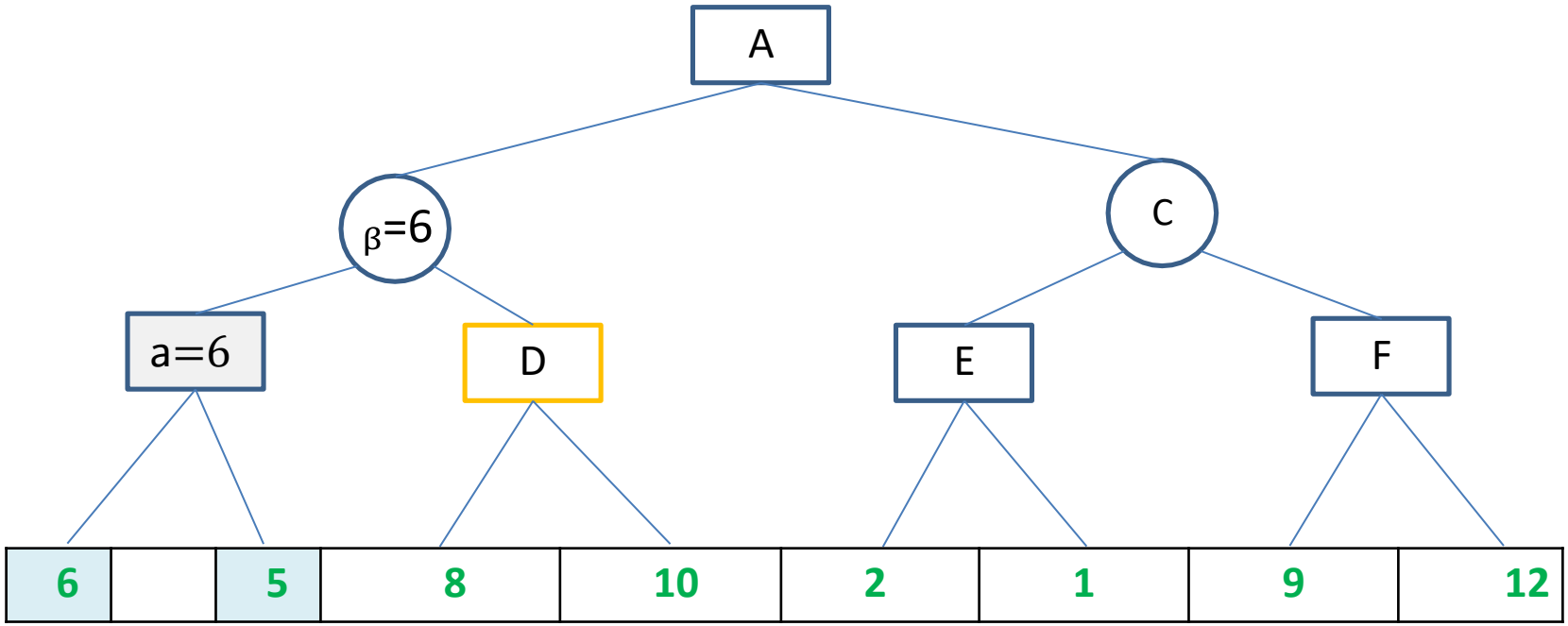
Alpha – Beta Pruning Example



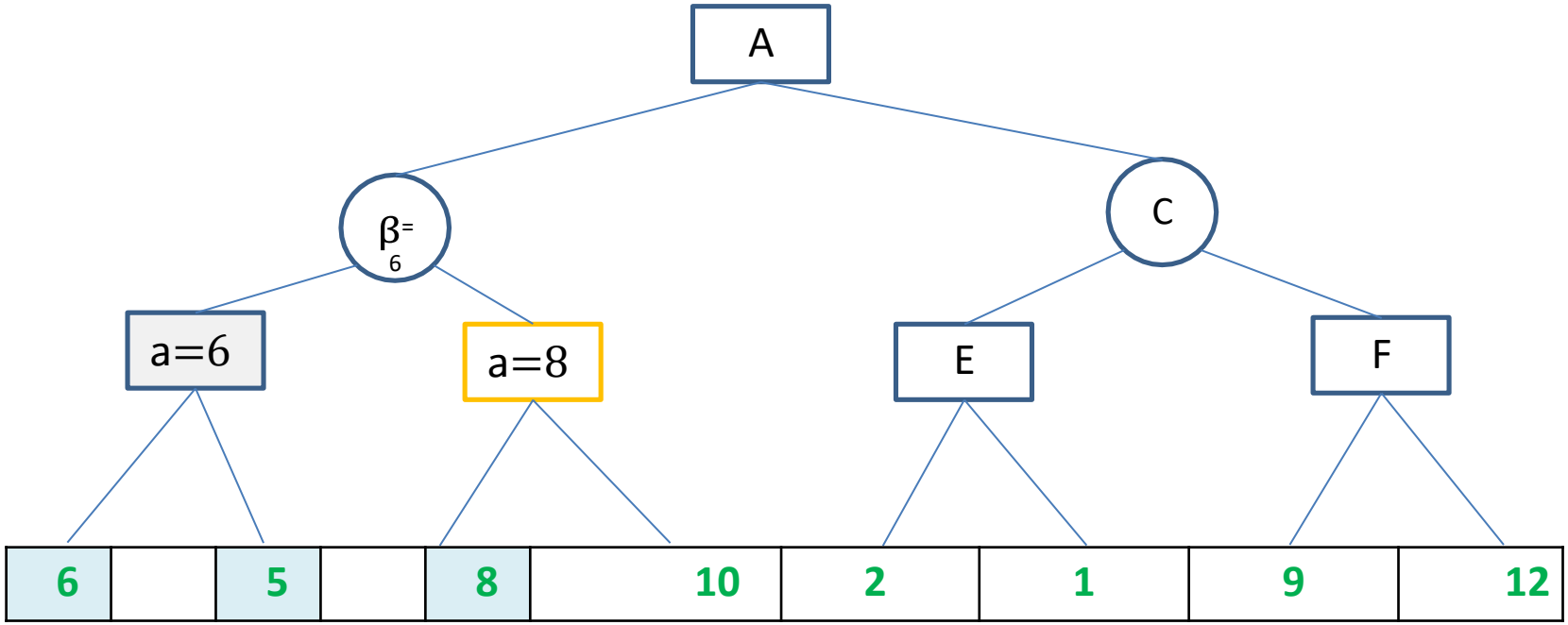
Alpha – Beta Pruning Example



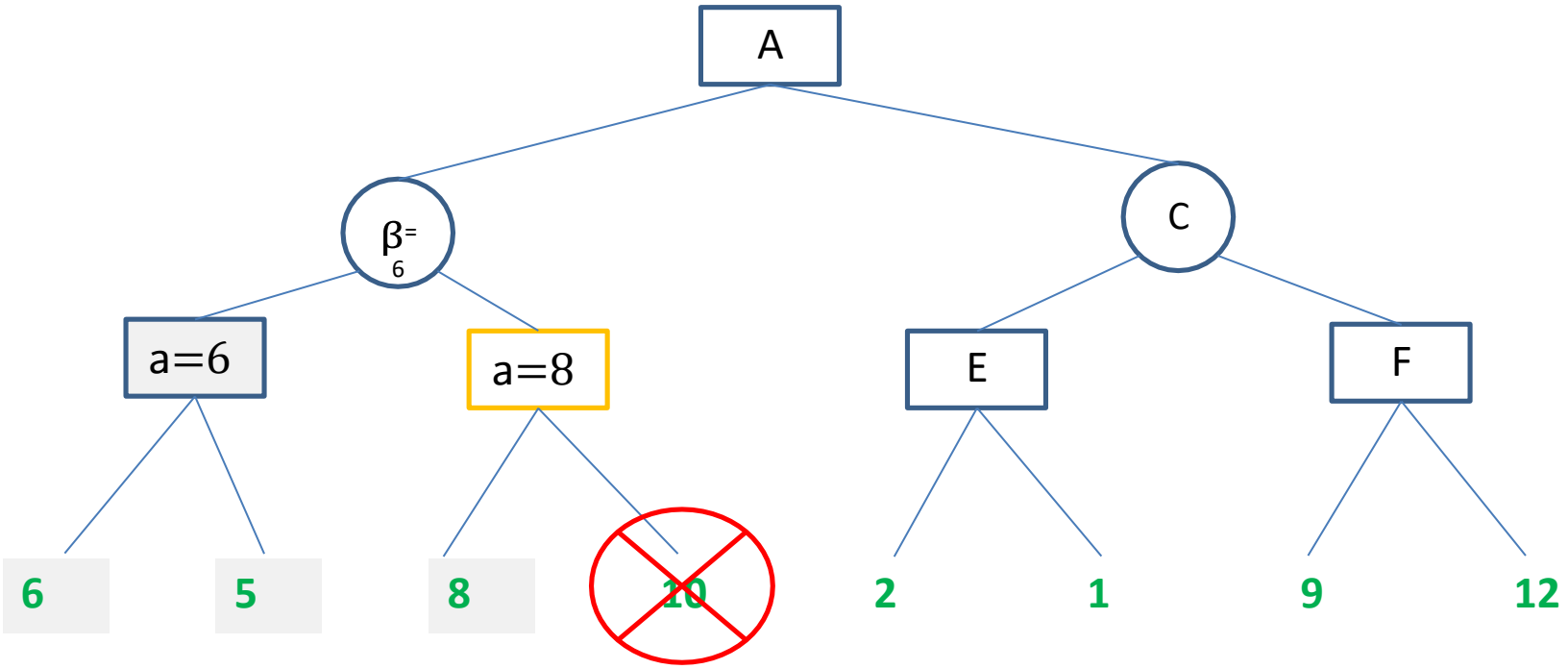
Alpha – Beta Pruning Example



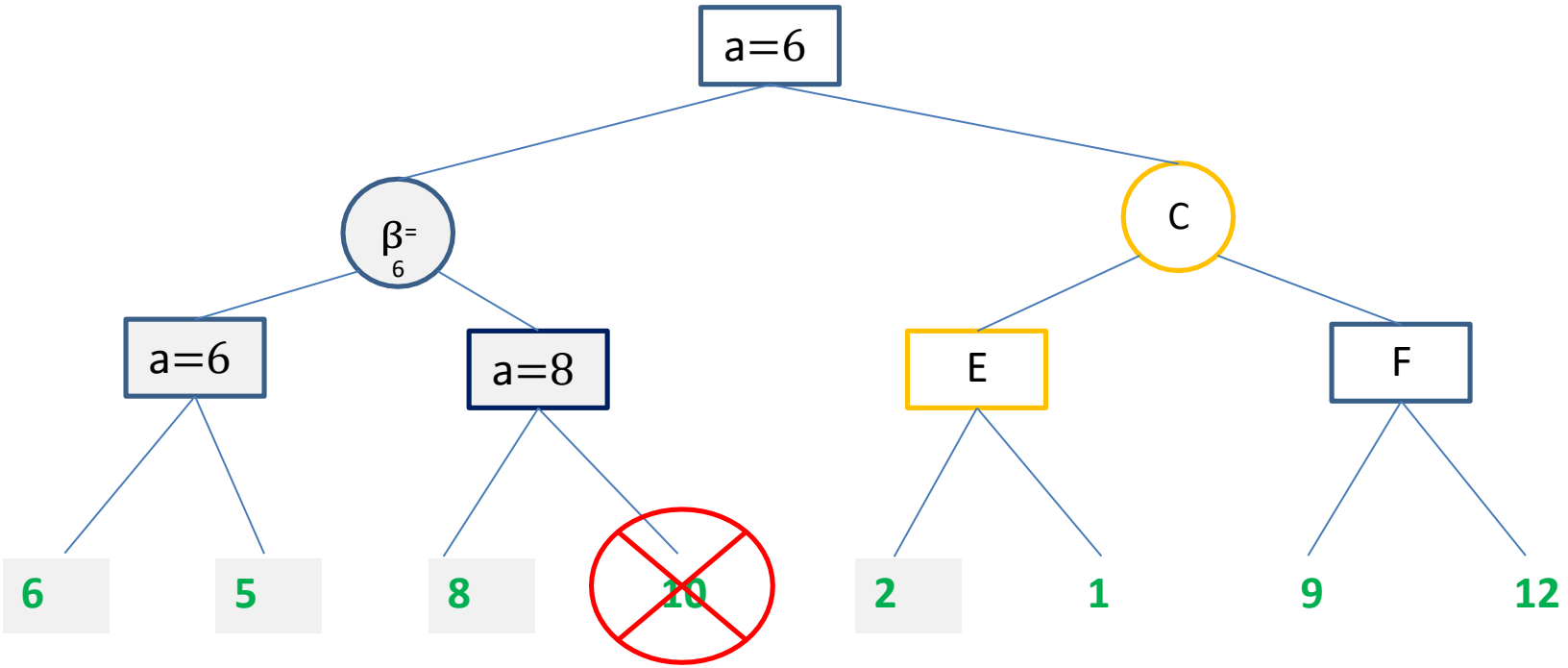
Alpha – Beta Pruning Example



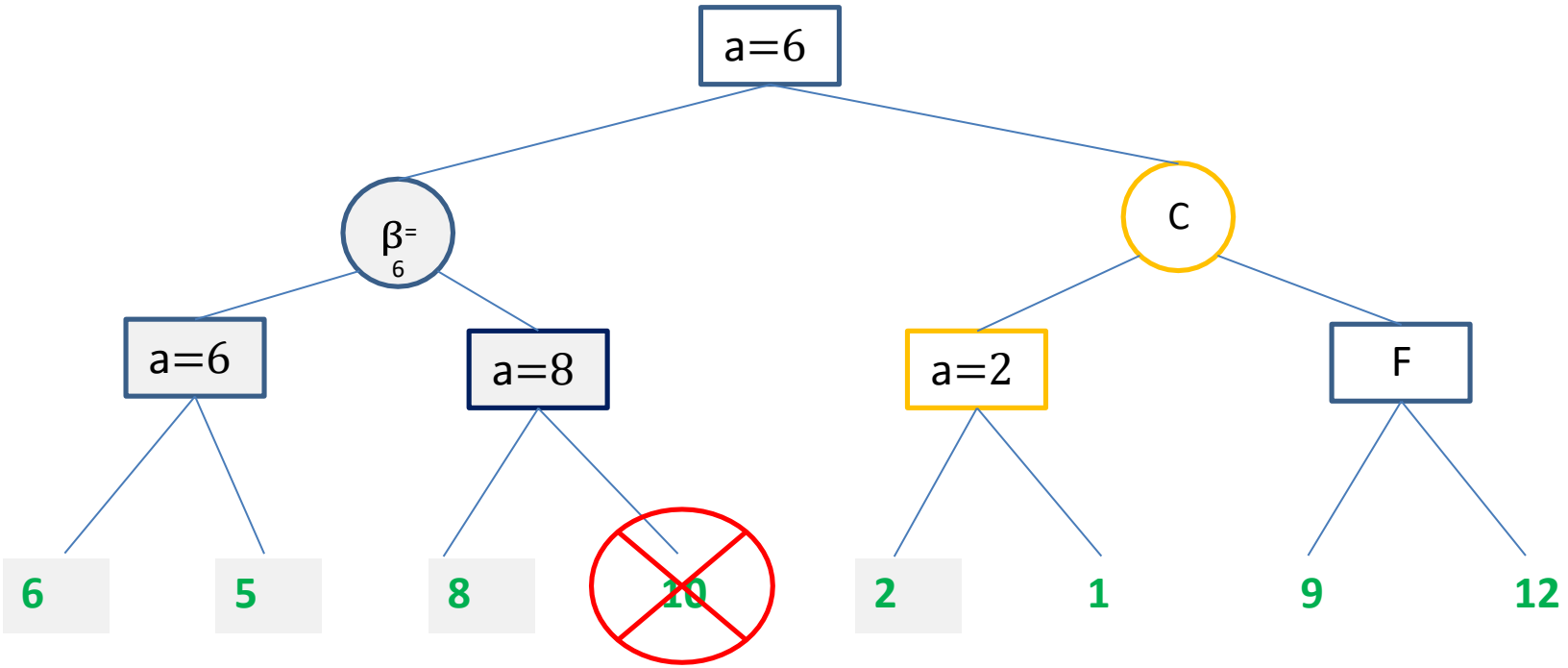
Alpha – Beta Pruning Example



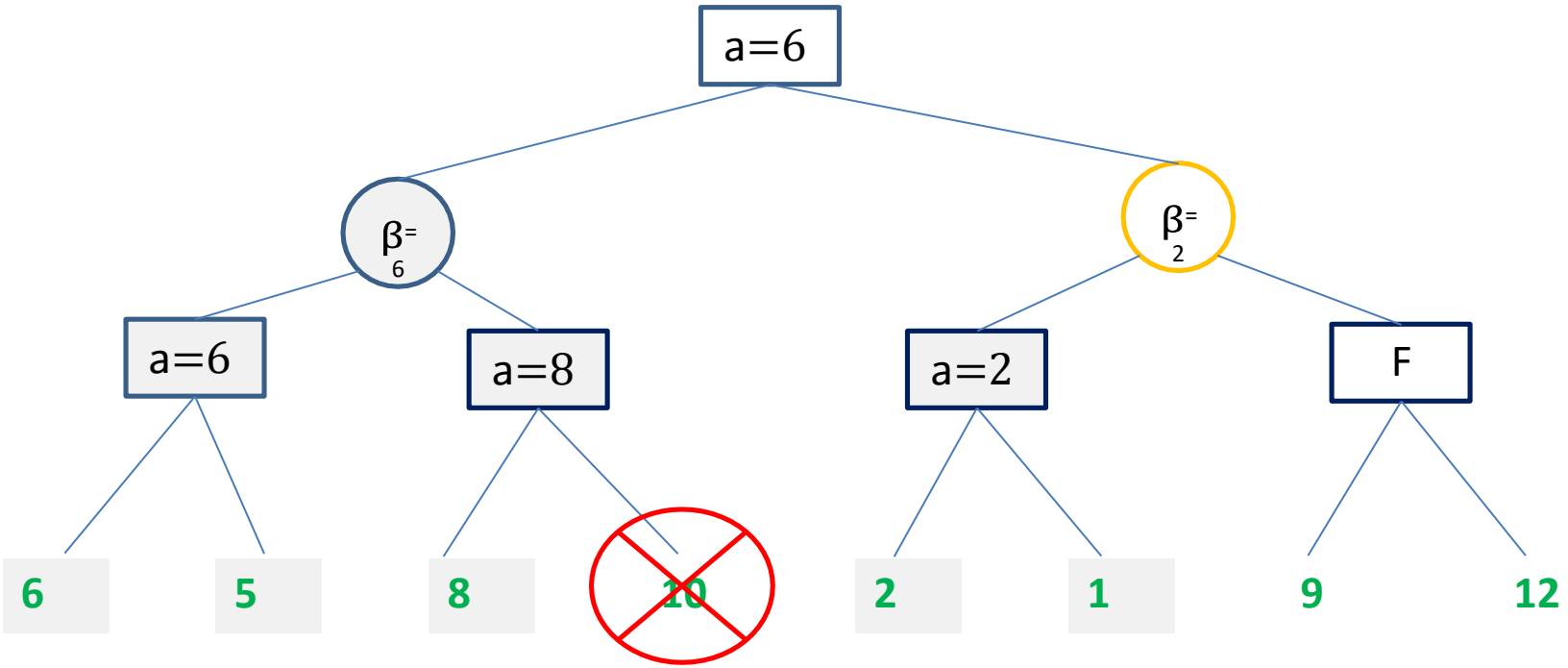
Alpha – Beta Pruning Example



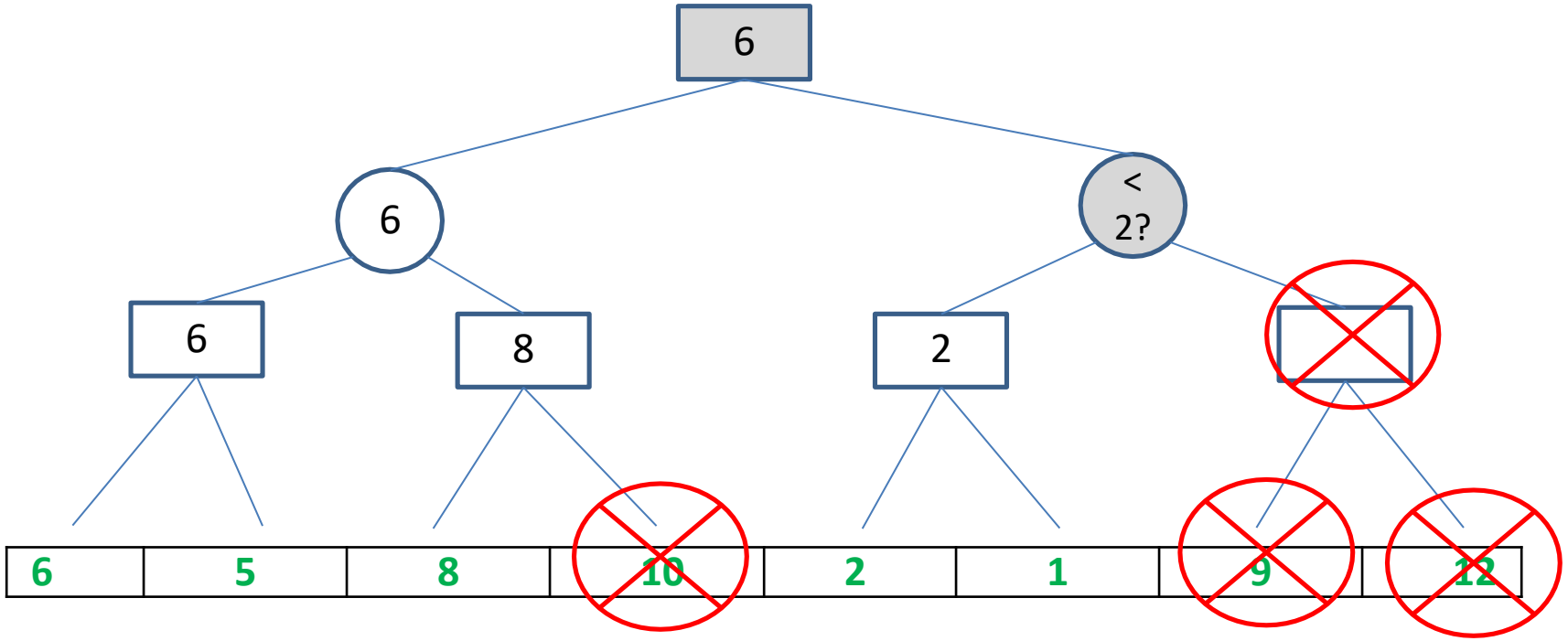
Alpha – Beta Pruning Example



Alpha – Beta Pruning Example



Alpha – Beta Pruning Example



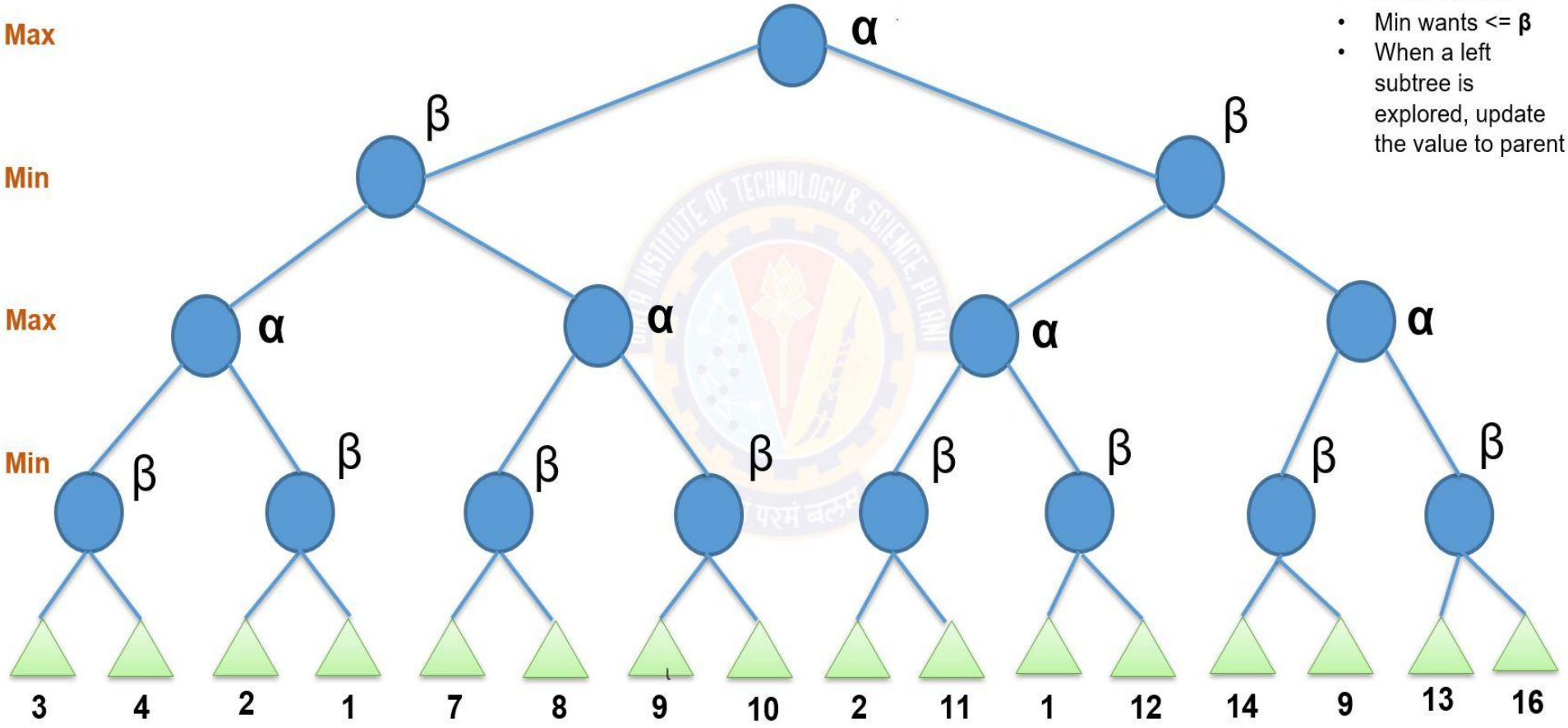
Alpha – Lower bound of Maximizer's value. Perceived value that Maximizer hopes to against a competitive Minimizer

Alpha – Beta Pruning Example 2!

innovate achieve lead

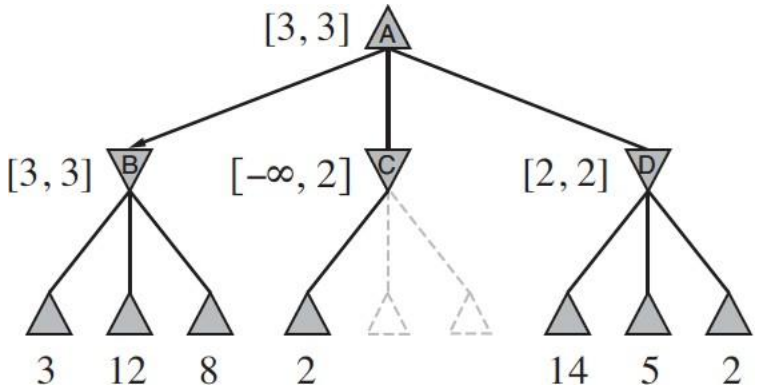
Remember

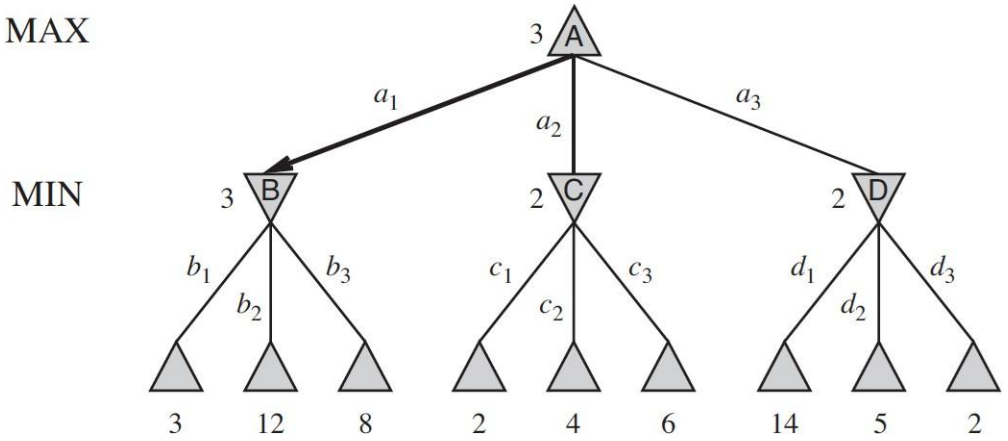
- Max wants >= **α**
- Min wants <= **β**
- When a left subtree is explored, update the value to parent



Computational Efficiency

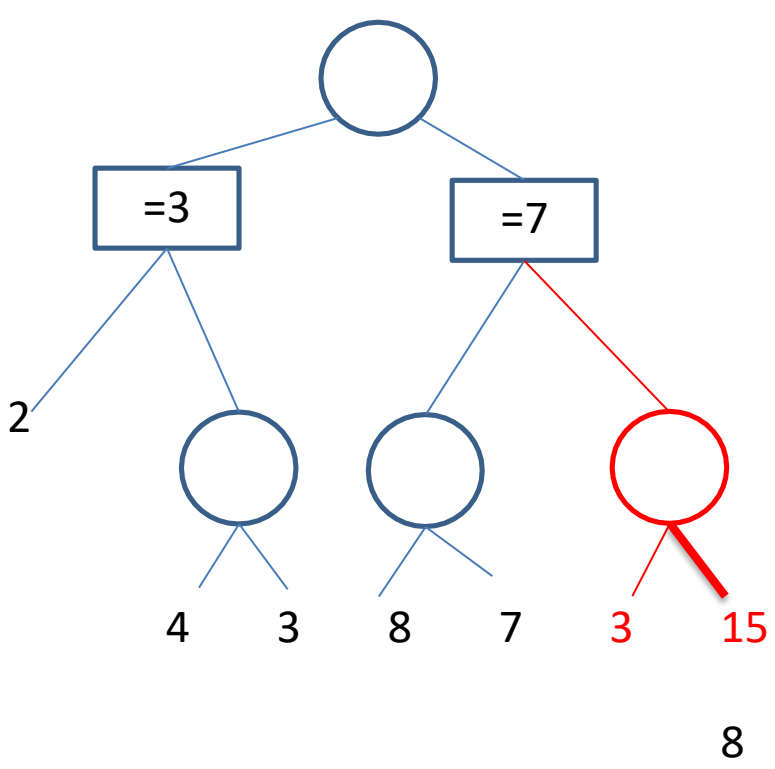
How to reduce the move generations better along while doing Alpha-Beta Pruning?



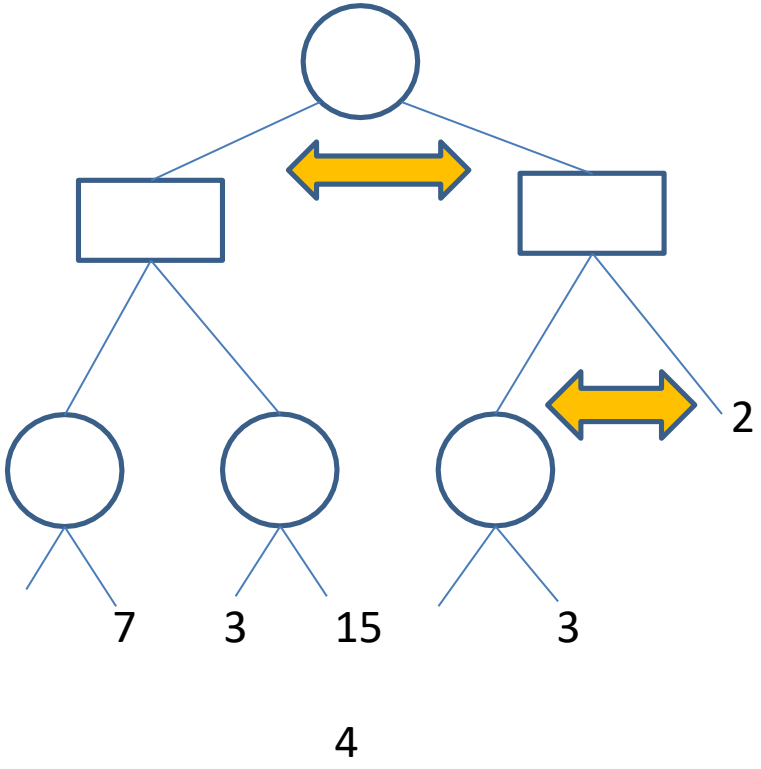


Computational Efficiency

After Move Ordering



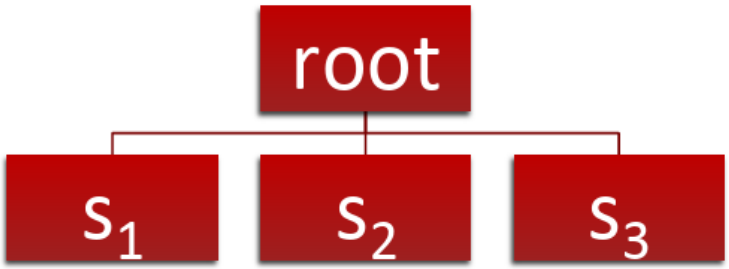
Before Move Ordering



Monte Carlo Tree Search

- Monte Carlo tree search (MCTS) is a heuristic search algorithm for decision processes, most notably those employed in software that plays board games.
- It's a probabilistic and search algorithm that combines classic tree search implementations alongside machine learning principles of reinforcement learning.
- Definition: Monte Carlo tree search is a heuristic search algorithm that relies on intelligent tree search to make decisions. It's most often used to perform game simulations, but it can also be utilized in cybersecurity, robotics, and text generation.

Example – Basic Monte Carlo Search

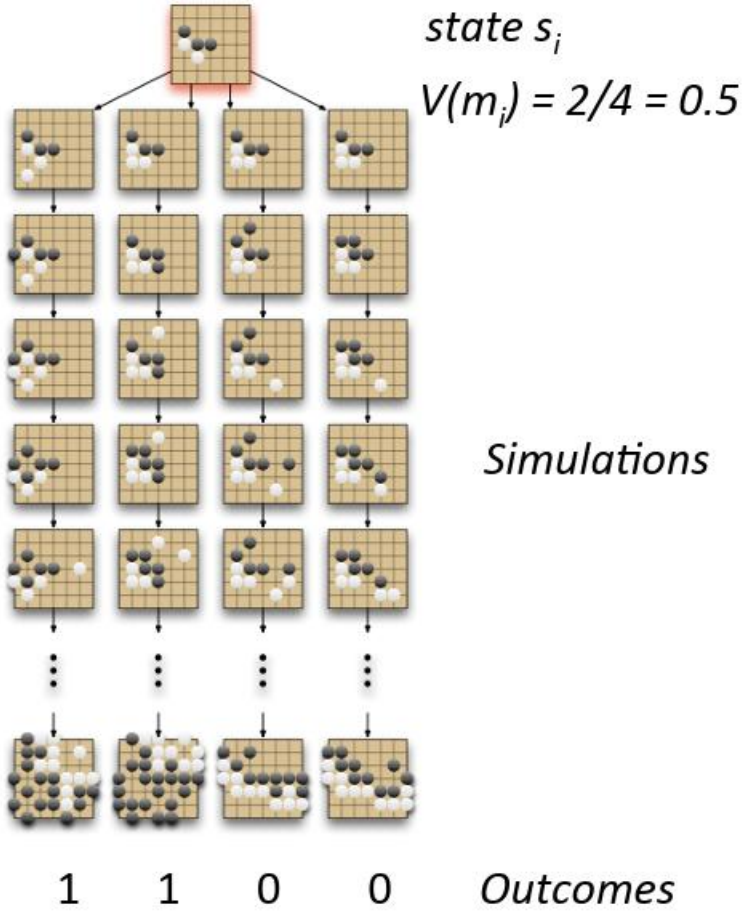


1 ply tree

root = current position

s_1 = state after move m_1

s_2 = ...

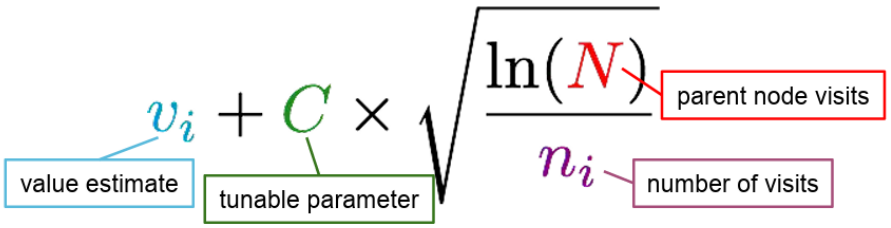


Naïve Approach

- Use simulations directly as an evaluation function for αβ
- Problems
  - Single simulation is very noisy, only 0/1 signal
  - Running many simulations for one evaluation is very slow
    - Example: Typical speed of chess programs 1 million eval/second
    - Go: 1 million moves/second, 400 moves/simulation, 100 simulations/eval = 25 eval/second
- Result: Monte Carlo was ignored for over 10 years in Go.

Monte Carlo Tree Search

- Idea: Use results of simulations to guide growth of the game tree
- Exploitation: focus on promising moves
- Exploration: focus on moves where uncertainty about evaluation is high
- Pick each node with probability proportional to:



  - Probability is decreasing in the number of visits (explore)
  - Probability is increasing in a node's value (exploit)
  - Always tries every option once

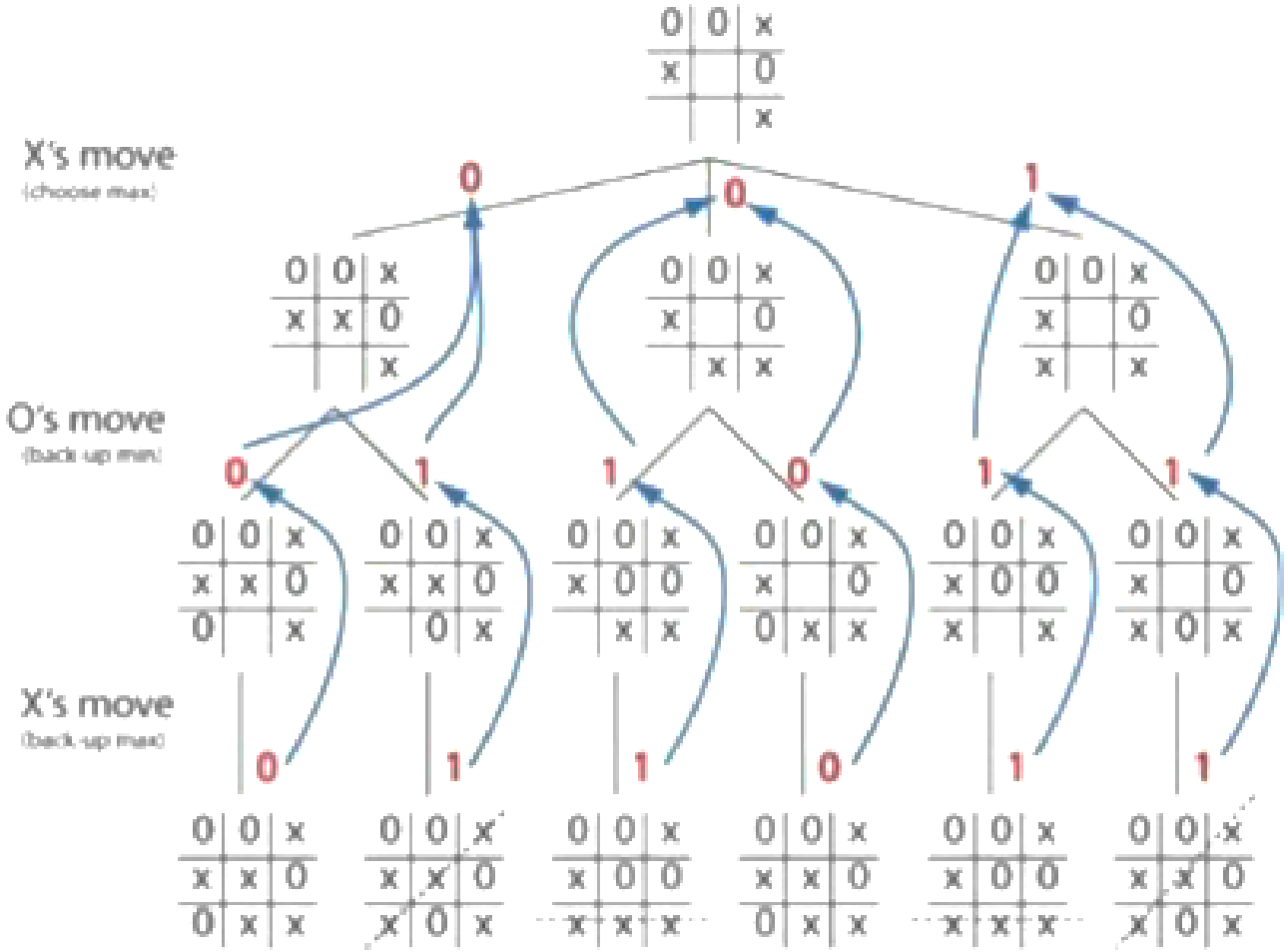
Monte Carlo Tree Search

- Monte Carlo tree search only searches a few layers deep into the tree and prioritizes which parts of the tree to explore.
- It then simulates the outcome rather than exhaustively expanding the search space.
- In doing so, it limits how many evaluations it has to make.
- The individual evaluation relies on the playout/simulation in which the algorithm effectively plays the game from a given starting point all the way to the leaf state by making completely random decisions, and then it records the results which is then used to update all the nodes in the random path all the way to the root.
- When it completes the simulation, it then selects the state that has the best rollout score.

MCTS for Combinatorial Game

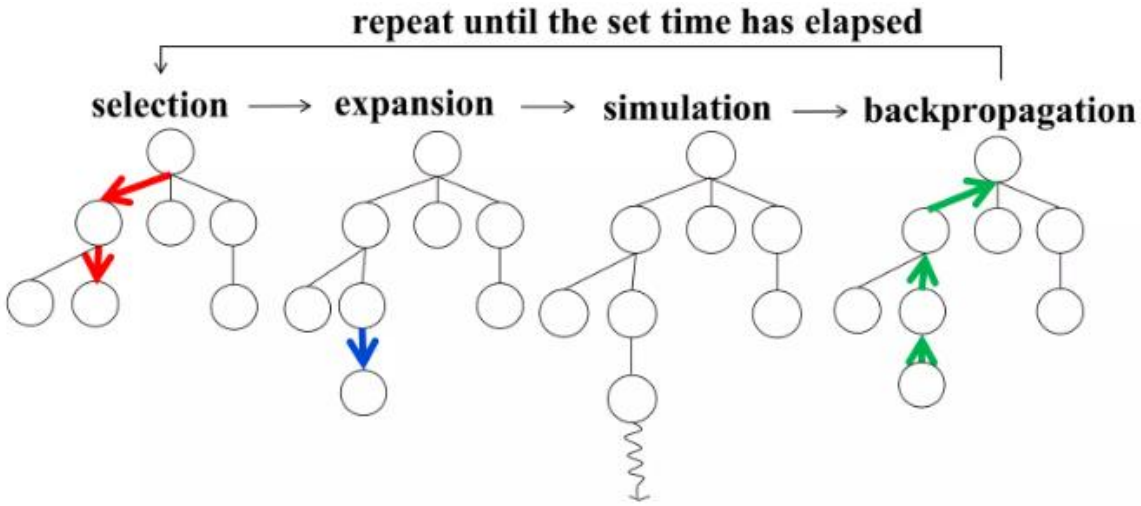
- MCTS is used in combinatorial games.
- These are sequential games with perfect information.
- The preconditions for a combinatorial game include:
- It must be a two-player game
- It must be a sequential game in which players take turns to play their move.
- It must have a finite set of well-defined moves.
- Examples: chess, checkers, Go and tic-tac-toe

Example Game Tree of Tic Tac Toe



How Monte Carlo Tree Search Works?

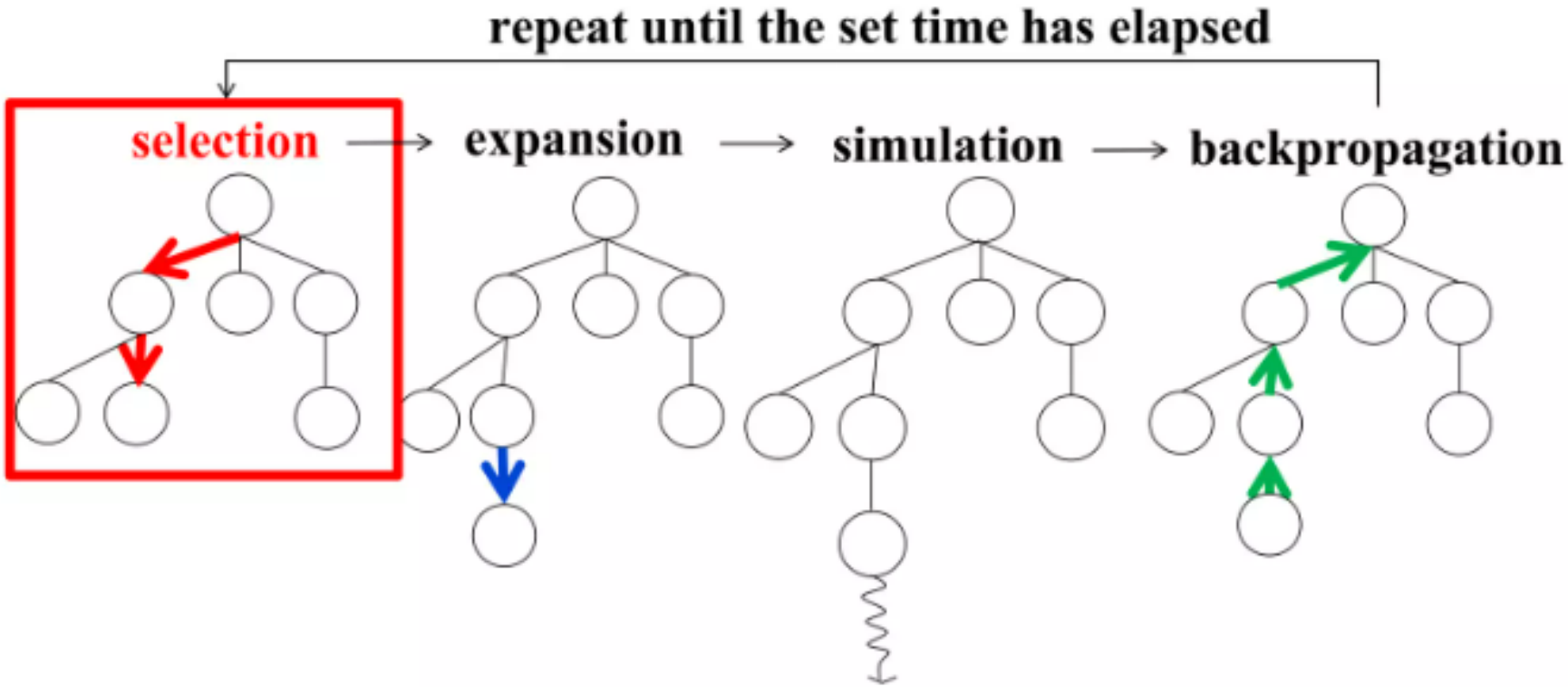
- The Monte Carlo tree search algorithm has four phases. We assign state values and number of interactions for each node:
  - Selection
  - Expansion
  - Simulation
  - Backpropagation



Selection

- **Idea:** Start from the root and choose the most promising child node repeatedly.
- **How:** Use a balance between:
  - Exploitation: node with high win rate
  - Exploration: node tried fewer times
- **Example:**
  In Tic-Tac-Toe, from the current board, the algorithm chooses the move with good past results but may still try less-explored moves.

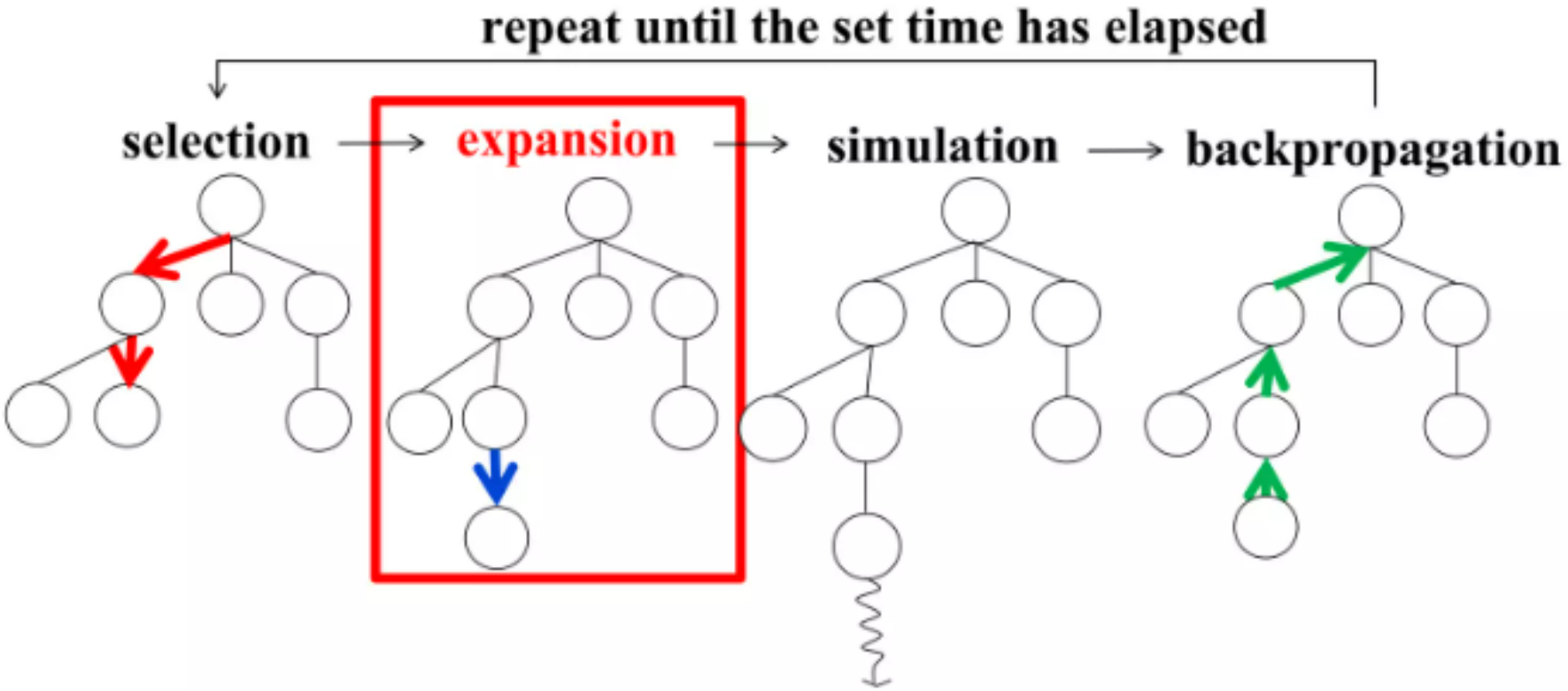
Selection


repeat until the set time has elapsed
selection
expansion
simulation
backpropagation

Expansion

- After selection, MCTS reaches a node where not all possible moves have been explored.
- Expansion means adding a new child node for one of the unexplored legal moves.
- This helps the search tree gradually grow over time instead of generating the entire tree at once.
- In Tic-Tac-Toe, assume the selected board state still has three legal moves:
  - Place X in top-right
  - Place X in middle-left
  - Place X in bottom-center
- MCTS chooses one unexplored move, such as **top-right**, and adds it as a new node in the tree.
- **Before expansion:**
  The selected node has only two explored children.
- **After expansion:**
  A new child node is added for the top-right move.

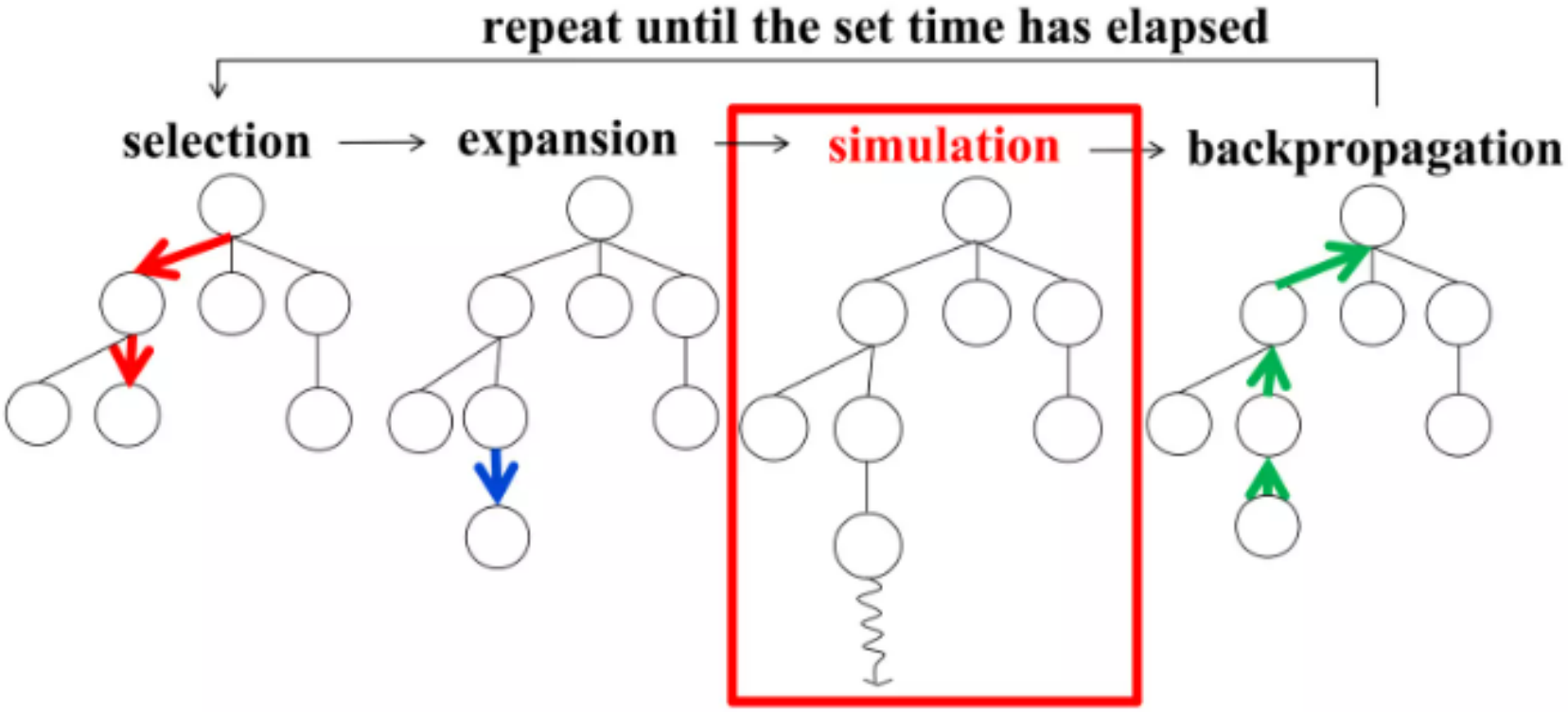
Expansion


repeat until the set time has elapsed
selection
expansion
simulation
backpropagation

Simulation

- Simulation is also called **rollout**.
- From the newly expanded node, the algorithm plays the game until the end.
- The moves during simulation may be:
  - Random
  - Based on simple rules
  - Based on a lightweight heuristic
- The purpose is not to play perfectly, but to estimate whether the new move may lead to a win, loss, or draw.

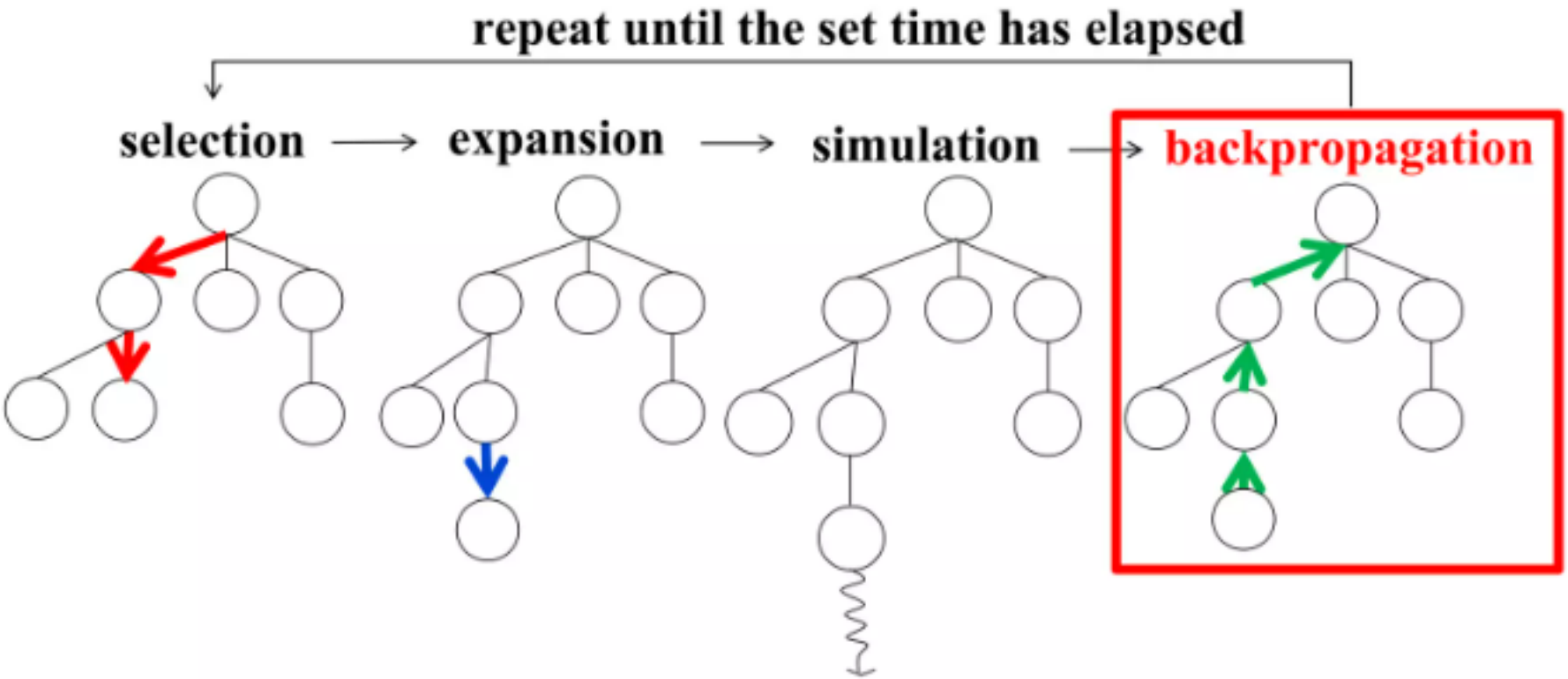
Simulation


repeat until the set time has elapsed
selection
expansion
simulation
backpropagation

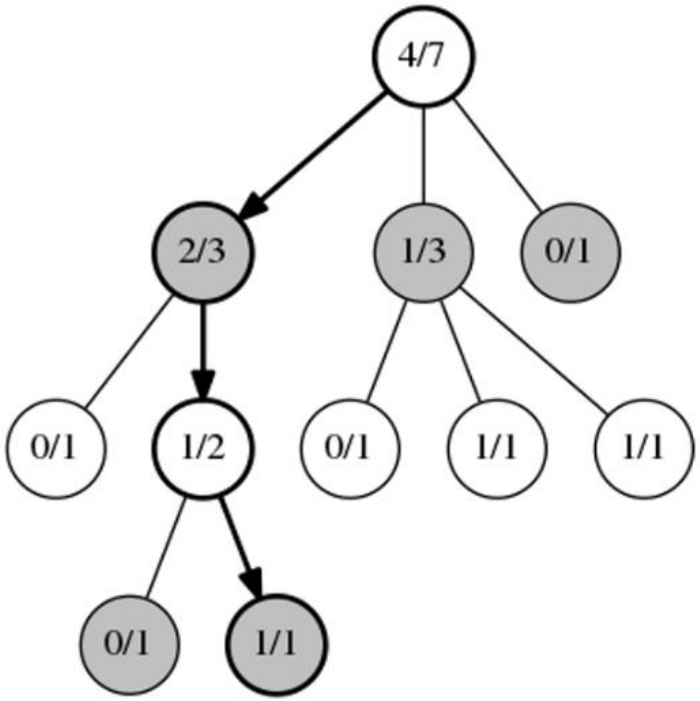
Backpropogation

- Once the simulation ends, the result is sent back through all nodes visited in that iteration.
- Each node on the path is updated.
- Usually, MCTS updates:
  - Visit count
  - Win score
  - Average value or win rate

Backpropogation

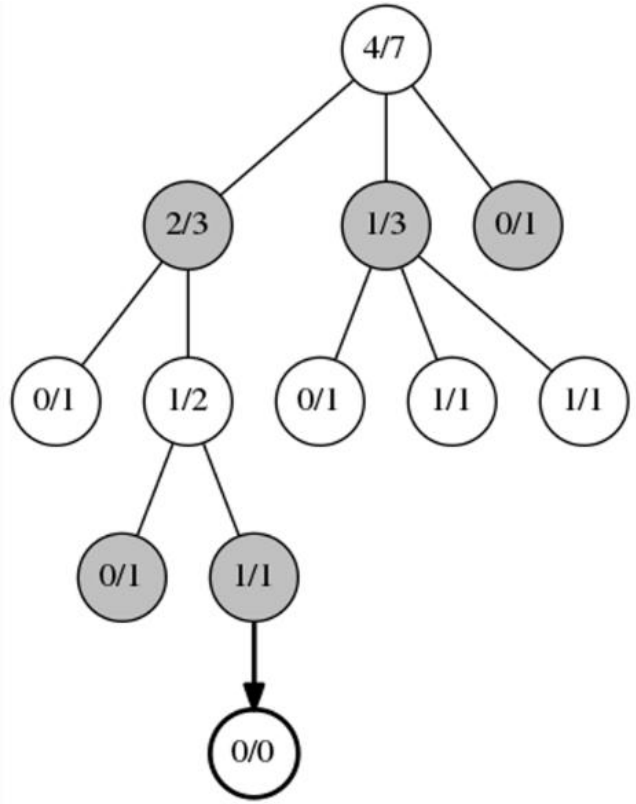


Selection – Expansion



Selection

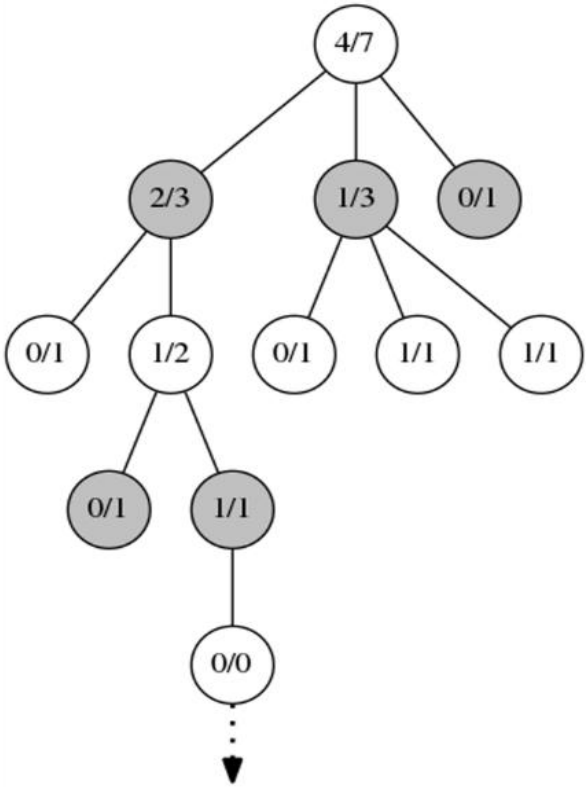
Here the positions and moves selected by the UCB1 algorithm at each step are marked in bold. Note that a number of playouts have already been run to accumulate the statistics shown. Each circle contains the number of wins / number of times played.



Expansion

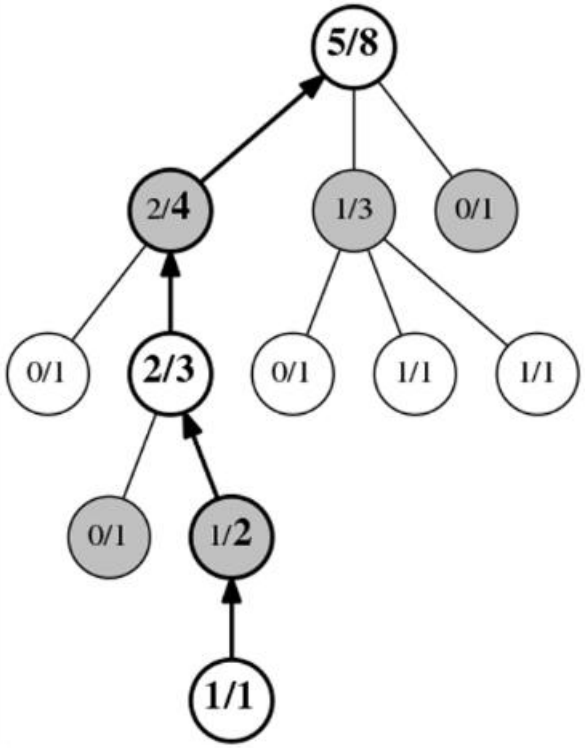
The position marked 1/1 at the bottom of the tree has no further statistics records under it, so we choose a random move and add a new record for it (bold), initialized to 0/0.

Simulation – Backpropogation



Simulation

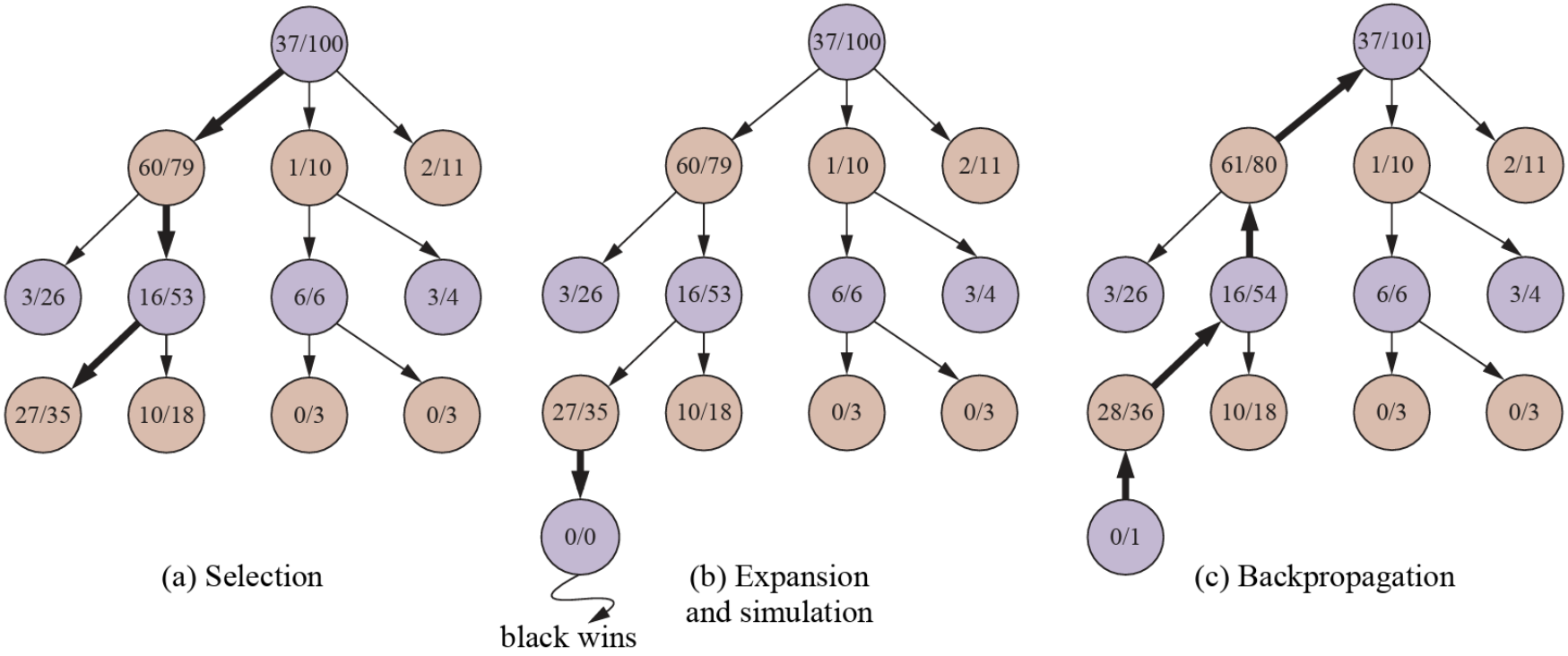
Once the new record is added, the Monte Carlo simulation begins, here depicted with a dashed arrow. Moves in the simulation may be completely random, or may use calculations to weight the randomness in favor of moves that may be better.



Back-Propagation

After the simulation reaches an end, all of the records in the path taken are updated. Each has its play count incremented by one, and each that matches the winner has its win count incremented by one, here shown by the bolded numbers.

Example 2



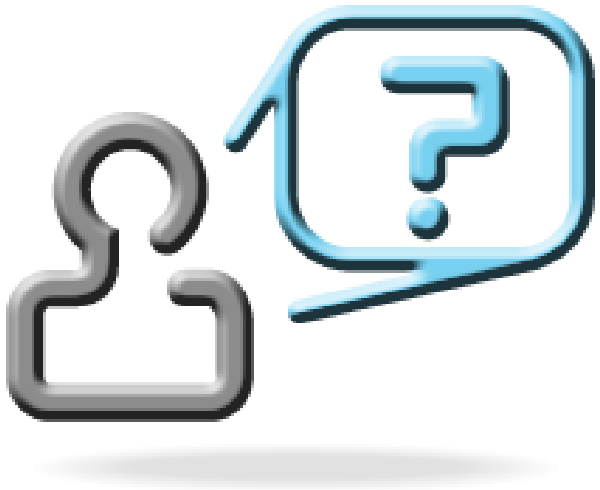
Monte Carlo Tree Search

```
function MONTE-CARLO-TREE-SEARCH(state) returns an action
    tree ← NODE(state)
    while IS-TIME-REMAINING() do
        leaf ← SELECT(tree)
        child ← EXPAND(leaf)
        result ← SIMULATE(child)
        BACK-PROPAGATE(result, child)
    return the move in ACTIONS(state) whose node has highest number of playouts
```

Mid-Semester (EC2) Examination

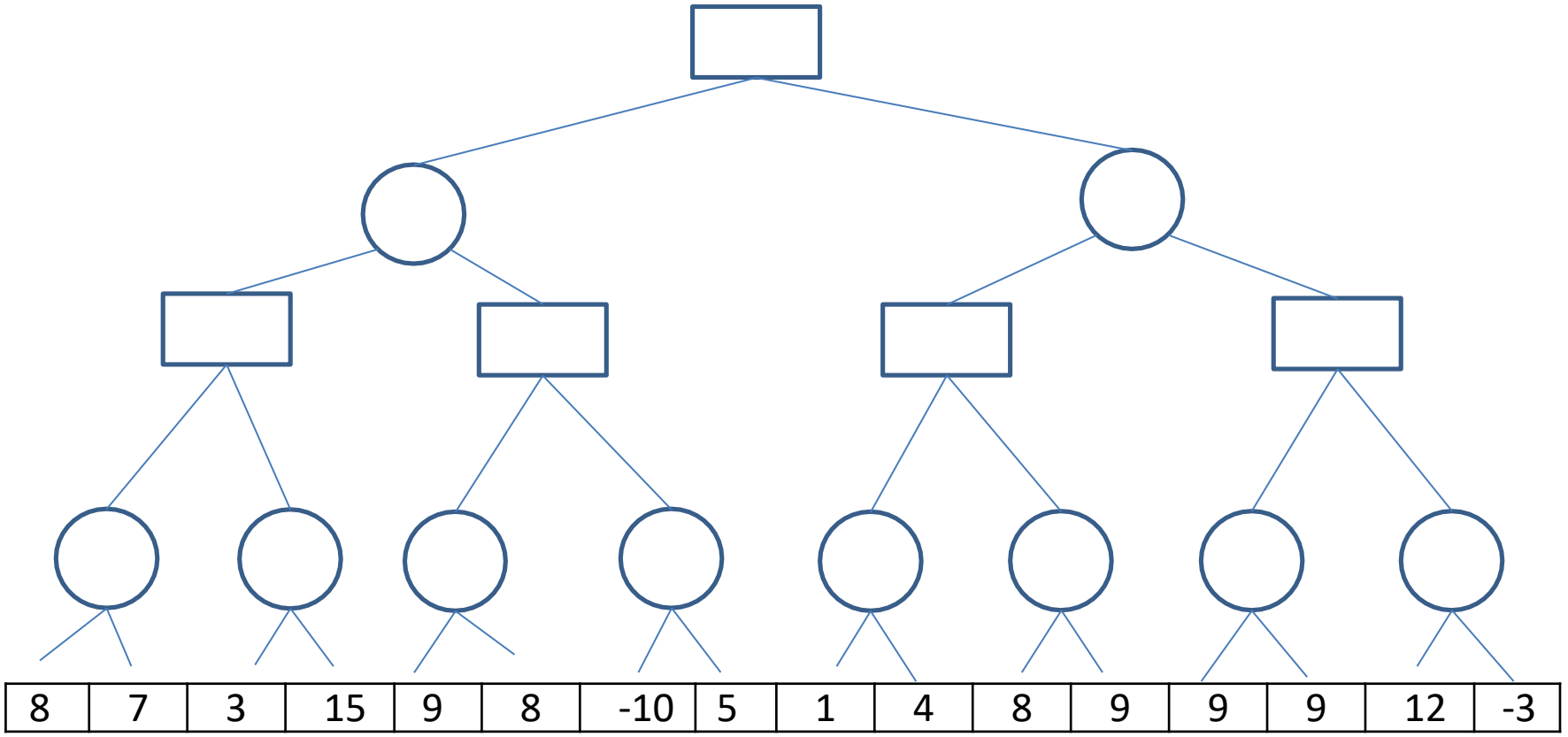
innovate achieve lead

- Topics: CS1 to CS8
- Duration: 2 hours
- Marks: 30 marks
- Type: Subjective
- Mode: Online in Exam centre
- Detailed syllabus and Sample paper will be available today!



Exercise 1

- Alpha – Lower bound of Maximizer's value. Perceived value that Maximizer hopes to get with a competitive Minimizer
- Beta – Upper Bound of Minimizer's value.



Sample Problem

Consider a 2-player game as in below diagram. The sliding-tile game consists of three “A” tiles, three “B” tiles, and two empty spaces. [Tile shaded in gray – Empty Space/Tile]. In this game a player wins if a pattern of player owned similar three tile are placed adjacent to each other. The term adjacency represents coins are in same row or same column. Assume the player with tile ‘A’ starts the move from this state. In the game, player “A” and “B” alternate in their turn.

•The puzzle has two legal moves: which results in the swap of position of the tile with the empty location

•Move #1: A tile may move into an adjacent empty location.

•Move #2: A tile can only hop over one tiles in the same row into the empty position.

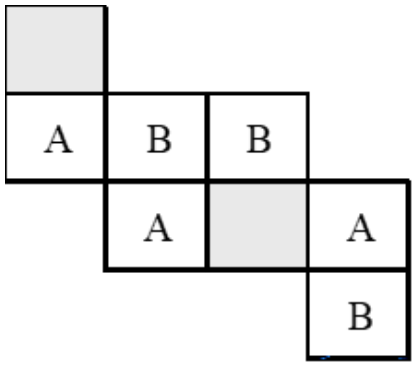
•Expand the complete game tree (with neat diagram) from the given current state up to exactly 3 levels.

•Calculate the utility of the leaves of the tree with below static evaluation function.

•Utility value = [3*(MAX’s win chance – MIN’s win chance)] + [2*(MAX’s No.of.adjacent.pairs – MIN’s No.of.adjacents.pairs)]

•Note: eg., For the given Initial configuration, MAX’s win chance is calculated as “In the current initial state 3 MAX coins ie., (A) are adjacent to empty cell and hence its win’s chance = 3.” and the MIN’s win chance is calculated as “In the current initial state 1 MIN coins ie., (B) is adjacent to empty cell and hence its win’s chance = 1.”

Apply the MIN MAX algorithm on the game tree



ACI: Disclaimer and Acknowledgements

- Few content for these slides may have been obtained from prescribed books and various other source(s) on the Internet.
- We hereby acknowledge all the contributors for their material and inputs and gratefully acknowledge people others who made their course materials freely available online. We have provided source information wherever necessary.
- We have added and modified the content to suit the requirements of the class dynamics & live session's lecture delivery flow for presentation.
- This *is not a full-fledged* reading materials. Students are requested to refer to the textbook w.r.t detailed content as per the course handout as shared over e-learning portal - Taxila.
- **Slide Source / Preparation / Review:**
- From BITS Pilani WILP: Prof. Rajavadhana, Prof. Indumathi, Prof. Sangeetha and Prof Parthasarathy PD, Prof Ramya Devi
- From BITS On-campus & External : Mr.Santosh GSK

Thank You for your time & attention !